# React Interview Prep

## Topic 9 — React.memo: Preventing Unnecessary Re-renders

**BEGINNER → INTERMEDIATE → EXPERT**

| | |
|---|---|
| ■ Covers | React.memo · Shallow comparison · 3 scenarios · Custom comparator · When NOT to use |
| ■ Series | React Interview Prep — Session 9 |

by Akshay Chavhan • React Interview Series

## 1. The Problem React.memo Solves

By default, every time a parent re-renders, ALL child components re-render too — even if their props didn't change.

```
// Parent re-renders on count change
// Child re-renders too — even though name='Akshay' never changed ■
function Child({ name }) {
console.log('Child rendered!'); // fires on every parent re-render
return <p>Hello {name}</p>;
}
```

```
// ■ Wrap in React.memo — skips re-render if props same
const Child = React.memo(function Child({ name }) {
console.log('Child rendered!'); // fires only when name changes
return <p>Hello {name}</p>;
});
```

> ■ ***React.memo wraps a component and memoizes its output. Same props → skip re-render → reuse last output.***

## 2. How React.memo Works — Shallow Comparison

**Parent re-renders** → **React.memo checks props** → **Props same? SKIP ■** / **Props changed? RE-RENDER**

| Prop type | Compared by | Works with React.memo? |
|---|---|---|
| string, number, boolean | Value === | ■ Yes — always safe |
| Object { } | Reference === | ■ New ref every render = always 'changed' |
| Array [ ] | Reference === | ■ New ref every render = always 'changed' |
| Function () => {} | Reference === | ■ New ref every render = always 'changed' |

> ■ ***Shallow = checks each prop with ===. Primitives work fine. Objects/arrays/functions must be stabilized with useMemo / useCallback.***

## 3. The 3 Scenarios

### Scenario 1 — Primitive props ■ Works perfectly

```
const Child = React.memo(({ name, count }) => <p>{name}: {count}</p>);
// name='Akshay' count={5} — never change → Child never re-renders ■
```

### Scenario 2 — Object prop ■ Needs useMemo

■ Wrong

```
// New object every render
const user = { name: 'Akshay' };
// React.memo sees new ref
// re-renders every time ■
return <Child user={user} />;
```

■ Correct

```
// Stable reference with useMemo
const user = useMemo(() => ({
name: 'Akshay'
}), []);
// React.memo sees same ref ■
return <Child user={user} />;
```

## Scenario 3 — Function prop ■ Needs useCallback

■ Wrong

```
// New function every render
const handleClick = () => {
console.log('clicked');
};
// React.memo re-renders ■
```

■ Correct

```
// Stable reference with
useCallback
const handleClick = useCallback(
() => console.log('clicked'),
[]
); // React.memo works ■
```

■ ***React.memo + useMemo + useCallback = the trio. memo checks props, useMemo stabilizes objects, useCallback stabilizes functions.***

## 4. Custom Comparison Function

Pass a second argument to React.memo to define exactly what 'props changed' means.

```
const Child = React.memo(
({ user }) => <p>{user.name}</p>,
(prevProps, nextProps) => {
// true → skip re-render (props 'equal')
// false → re-render (props 'different')
return prevProps.user.id === nextProps.user.id;
// only re-render if user ID changes — ignore other fields
}
);
```

> ■■ ***Be careful — if you forget to compare a prop, the component shows stale data.***

## 5. React.memo vs useMemo vs useCallback

| Hook / API | Memoizes | Use for | Needs partner? |
|---|---|---|---|
| React.memo | A component | Skip child re-render if props same | useMemo + useCallback |
| useMemo | A value | Stable object/array ref, expensive calc | React.memo on child |
| useCallback | A function | Stable function ref passed as prop | React.memo on child |

## 6. When NOT to Use React.memo

| Situation | Use React.memo? | Why |
|---|---|---|
| Cheap component (e.g. ) | ■ No | memo overhead costs MORE than re-render |
| Props change on every render | ■ No | memo never skips — pure wasted overhead |
| Parent never re-renders | ■ No | No problem to solve in the first place |
| Component has heavy render output | ■ Yes | Skipping expensive renders = real gain |
| Stable props + frequent parent updates | ■ Yes | Child protected from parent noise |
| Can measure problem in Profiler | ■ Yes | Always optimise based on evidence |

> ■ ***Rule: measure first with React DevTools Profiler, then optimise. Never add React.memo blindly — it has its own cost.***

## 7. Interview Q&A;

Q Beginner **Q1. What is React.memo?**

➜ React.memo is a Higher Order Component that wraps a component and memoizes its output. If the component receives the same props as the previous render, React skips re-rendering and reuses the last output. It does a shallow comparison of props by default.

Q Beginner **Q2. What is shallow comparison?**

➜ Shallow comparison checks each prop using strict equality ===. Primitives like strings and numbers are compared by value and work correctly. Objects, arrays and functions are compared by reference — so even if content is identical, a new reference means React.memo thinks the prop changed and re-renders anyway.

Q Intermediate **Q3. React.memo isn't working — what's likely wrong?**

➜ Almost always it's an unstable prop reference. Either an object or array created inline without useMemo, or a function created without useCallback. Every render creates a new reference — React.memo sees it as changed and re-renders. Fix: stabilize objects with useMemo and functions with useCallback.

Q Intermediate **Q4. What is the difference between React.memo, useMemo and useCallback?**

➜ React.memo memoizes an entire component — skips re-rendering if props didn't change. useMemo memoizes a computed value inside a component. useCallback memoizes a function inside a component. They work together — useMemo and useCallback stabilize the props that React.memo then compares.

Q Intermediate **Q5. When should you NOT use React.memo?**

➜ When the component is cheap to render — React.memo overhead can cost more than the re-render it prevents. Also avoid it when props always change every render — memo never skips and only adds cost. Only use it when you can measure a real re-render performance problem in the Profiler.

Q Expert **Q6. What is the custom comparison function in React.memo?**

➜ React.memo accepts a second argument — a function that receives previous and next props and returns true to skip re-render or false to re-render. It lets you define exactly what 'props changed' means for that component. Be careful — if you miss comparing a prop, the component can show stale UI data.

Q Expert **Q7. How do you prove React.memo is working?**

➜ Use React DevTools Profiler. Record an interaction, then check the flame graph. Without memo, both parent and child show coloured bars meaning they rendered. With memo and same props, the child bar turns grey — meaning it was skipped. Grey = memoized and skipped. Coloured = actually rendered.